

บทที่ 6

Entity Bean

6.1 แนวคิด Persistent concept

6.1.1 Java Object Serialization

ในการทำงานกับออบเจกต์ที่เขียนด้วยภาษาจาวา อาจต้องการเก็บสถานะ (state) ของออบเจกต์ลงใน permanent storage วิธีหนึ่งที่ได้คือใช้วิธีการ object serialization ซึ่งเป็นการแปลงออบเจกต์ให้อยู่ในรูป สตรีมของไบนารีข้อมูล (byte stream) หลังจากนั้น สามารถจะทำอะไรกับไบนารีสตรีมนี้นี้ได้ เช่น ส่งข้ามระบบเครือข่าย หรือเก็บลง storage เช่น ไฟล์, ฐานข้อมูล, JNDI หรือ

เมื่อต้องการเรียกกลับขึ้นมาใช้งานใหม่ จะต้องทำในกระบวนการย้อนกลับ คือ อ่านไบนารีสตรีมขึ้นมา แล้วนำไปสร้างเป็นออบเจกต์ใหม่อีกครั้ง อย่างไรก็ตาม วิธีนี้ไม่เหมาะในงานที่มีการ query บ่อย เนื่องจาก operation ทุกอย่างทำกับออบเจกต์ จึงต้องโหลดออบเจกต์กลับขึ้นมาเพื่อดูว่าเงื่อนไขตรงกับที่ query หรือไม่

6.1.2 Object-Relational Mapping

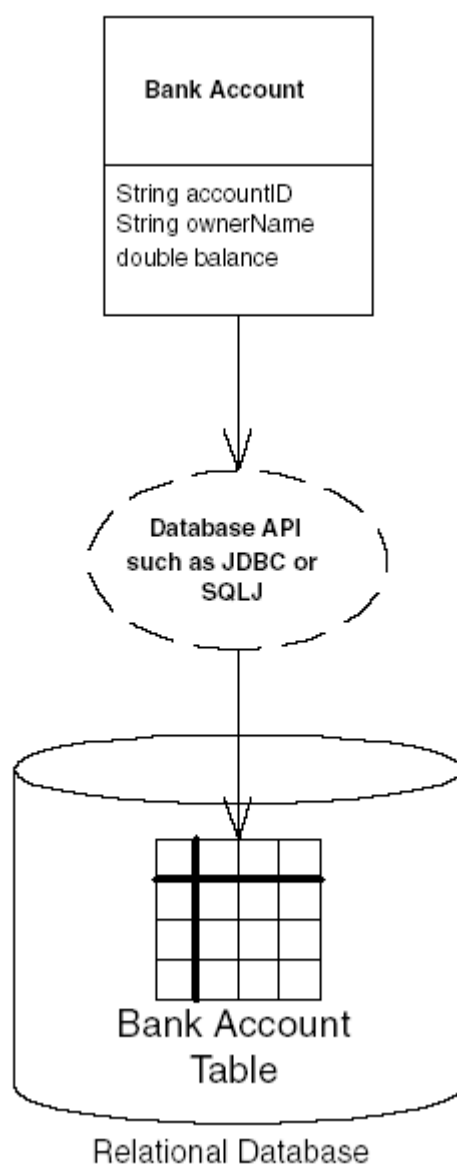
อีกวิธีหนึ่งที่ได้รับคามนิยมในการเก็บจาวาออบเจกต์ก็คือ การใช้ฐานข้อมูลเชิงสัมพันธ์ (relational database) เช่น Oracle หรือ Microsoft SQL Server แทนที่จะทำการ serialize ออบเจกต์ การบันทึกจาวาออบเจกต์ใช้ JDBC หรือ SQL/J เพื่อแมปข้อมูลของออบเจกต์ไปยังฐานข้อมูลเชิงสัมพันธ์ นอกจากนี้ยังสามารถบันทึกชื่อของจาวาคลาสที่สัมพันธ์กับข้อมูลนี้ เพื่อที่จะสร้าง (instantiate) คลาสที่ถูกต้องตามออบเจกต์ที่ต้องการอ่านกลับขึ้นมาในหน่วยความจำ ในการโหลดออบเจกต์จากฐานข้อมูล จะต้องสร้างออบเจกต์จากคลาสนั้น อ่านข้อมูลขึ้นมาจากฐานข้อมูล และเก็บค่าเหล่านั้นกับฟิลด์ของอินสแตนซ์ของออบเจกต์ที่อยู่ในหน่วยความจำ ดังแสดงในรูปที่ 6-1 และ 6-2

6.1.3 ฐานข้อมูลเชิงวัตถุ (Object Database)

Object Database Management System (ODBMS) สามารถเก็บออบเจกต์ไว้ในฐานข้อมูลได้โดยไม่ต้องผ่านกระบวนการ O/R mapping และทำให้การเขียนโค้ดเพื่อเข้าถึงข้อมูลทำได้ง่ายขึ้น โดยการเขียนโปรแกรมติดต่อกับ API ของฐานข้อมูลเชิงวัตถุแทนการเขียนโค้ดติดต่อกับ API ของฐานข้อมูลเชิงสัมพันธ์

ฐานข้อมูลเชิงวัตถุส่วนใหญ่ จะให้เครื่องมือในการ query ข้อมูลที่เรียกว่า Object Query Language (OQL) ซึ่งเป็นการ query ในระดับสูงกว่าฐานข้อมูลเชิงสัมพันธ์

นอกจากนี้ ระบบฐานข้อมูลเชิงวัตถุยังมีความถูกต้อง (integrity) และความปลอดภัย (security) สูงกว่าระบบฐานข้อมูลเชิงสัมพันธ์ ในกรณี persistent object มีความซับซ้อนมาก ๆ เช่น การเก็บฐานข้อมูลของ



รูปที่ 6-1 Object-Relational Mapping

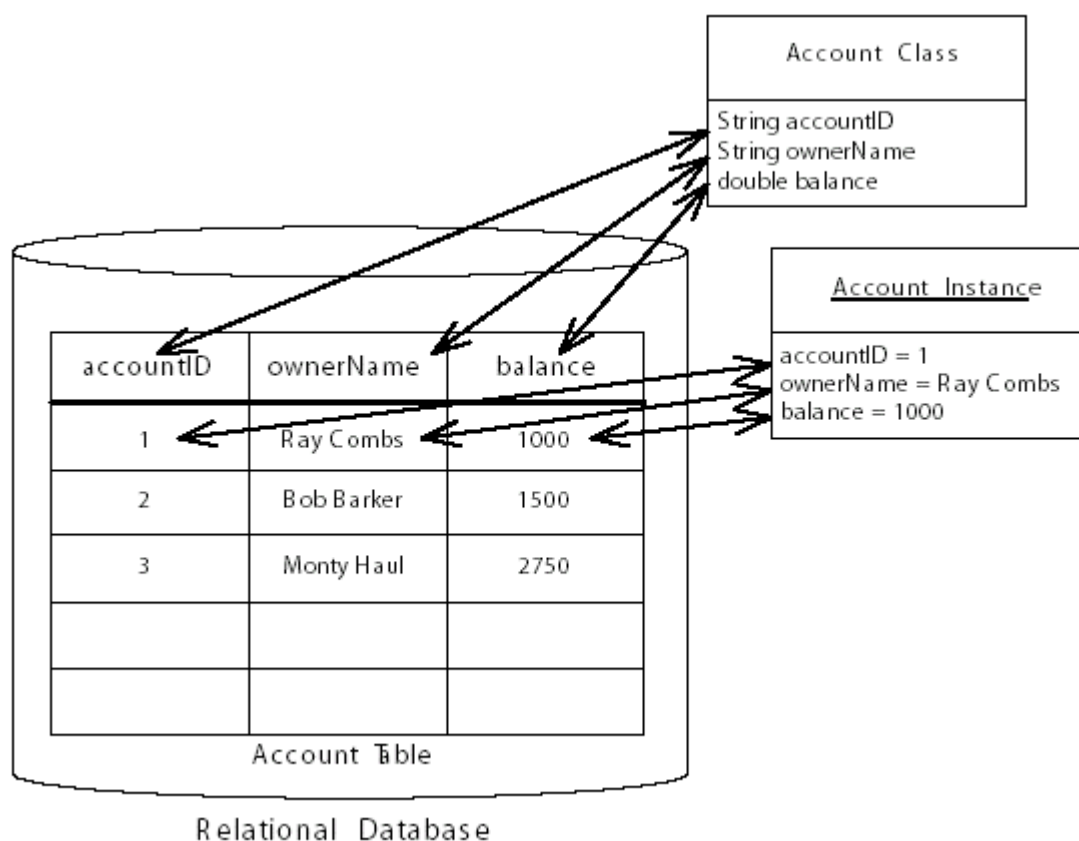
CAD/CAM แต่ถ้าออกแบบไม่ดีไม่ซับซ้อนและสามารถแมปไปเป็นฐานข้อมูลเชิงสัมพันธ์ได้ง่าย เช่น แอปพลิเคชันทางธุรกิจต่าง ๆ ซึ่งเน้นที่ปริมาณงานจำนวนมากแล้ว ฐานข้อมูลเชิงสัมพันธ์จะมีประสิทธิภาพที่สูงกว่า

6.2 แนวคิดของ Entity Bean

6.2.1 อินสแตนซ์ของ Entity Bean

คอมโพเนนต์ที่ถูกดีพลอยแบ่งแยกความแตกต่างได้ดังนี้

Application Logic Components - เป็นคอมโพเนนต์ที่ให้บริการสำหรับงานทั่ว ๆ ไป เช่น การคำนวณราคาค่าสั่งซื้อ



รูปที่ 6-2 ตัวอย่างการแมประหว่างออบเจกต์และฐานข้อมูลเชิงสัมพันธ์

การออกบัตรเครดิตของลูกค้า

การหาอินเวอร์สของเมตริก

โดยส่วนใหญ่คอมพิวเตอร์เหล่านี้จะแทนการกระทำ เหมาะกับการใช้กับบิซิเนสโพรเซส

Persistent Data Components - เป็นออบเจกต์ที่รู้ถึงวิธีการแปลงตัวมันเองลงใน persistent storage โดยใช้กลไกต่าง ๆ เช่น serialization , O/R mapping ไปเป็นฐานข้อมูลเชิงสัมพันธ์หรือฐานข้อมูลเชิงวัตถุ ออบเจกต์ชนิดนี้เป็นตัวแทนของข้อมูล (data) เช่น

ข้อมูลในบัญชีธนาคาร เช่น หมายเลขบัญชี และ ยอดคงเหลือ

ข้อมูลของฝ่ายบุคคล เช่น ชื่อ แผนก เงินเดือน

เรียกได้ว่าคอมพิวเตอร์ชนิดนี้ใช้แสดงถึงสิ่งที่เป็นค่านาม เช่น คน สถานที่ สิ่งของ

การมองข้อมูลเหล่านี้เป็นออบเจกต์ แทนที่จะจัดการในระดับข้อมูลดิบ (raw data) นั้น ทำให้จัดการได้ง่ายขึ้น สามารถรวมข้อมูลที่สัมพันธ์กันให้อยู่ในออบเจกต์เดียว สามารถใช้เมธอดเพื่อจัดการกับกลุ่มข้อมูลเหล่านี้ และได้รับการบริการจากมิดเดิลแวร์ เช่น ทรานแซกชัน , การเข้าถึงระบบเครือข่าย , และการรักษาความปลอดภัย

entity bean เป็นคอมโพเนนต์ประเภท persistent data component ที่รู้วิธีการแปลงตัวมันเองลงในแหล่งเก็บข้อมูล เช่น ฐานข้อมูล โดยเก็บอยู่ในรูปแบบของฟิลด์ (field) หลาย ๆ ฟิลด์ เช่น ฟิลด์ของหมายเลขบัญชี และฟิลด์ของยอดคงเหลือ ซึ่งจะมีเมธอดที่สัมพันธ์กันอยู่ภายในคอมโพเนนต์ด้วย เช่น getBankAccountNumber() และ getBankAccountBalance()

entity bean อาจเปรียบเสมือน serializable java object ที่สามารถเก็บตัวเองในรูปกลุ่มของบิตลงในแหล่งบันทึกข้อมูลที่ persistent entity bean สามารถทำ persistent ได้หลายวิธี เช่น serialization , O/R mapping และ object database persistence เนื่องจากข้อกำหนดของ Enterprise JavaBeans ไม่ได้บังคับวิธีการทำ persistent ไว้

ข้อแตกต่างระหว่าง entity bean กับ session bean คือ session bean แสดงถึงกระบวนการทำงานที่เกิดขึ้นโดยผู้ใช้ และจบลงเมื่อผู้ใช้จบการทำงานไป แต่ entity bean จะเก็บตัวข้อมูลจริง ๆ เช่น ข้อมูลของสินค้า, บัญชีธนาคาร, คำสั่งซื้อ หรือ ข้อมูลลูกค้า entity bean ไม่ทำงานที่ซับซ้อนมาก เช่น การออกไปเสร็จให้ลูกค้า แต่ตัวมันจะแทนลูกค้าเองเลย และ entity bean เป็นออบเจกต์ที่มีสถานะคงทน (persistent state object) แม้ว่าผู้ใช้จะจบการทำงานไปแล้วก็ตาม

อินสแตนซ์ของ entity bean จะมีลักษณะดังนี้

- เป็นสิ่งที่แทนข้อมูลที่ persistent ในลักษณะของภาษาจาวาที่อยู่ในหน่วยความจำ
- รู้วิธีอ่านค่าตัวเองเข้ามาจากแหล่งบันทึกข้อมูล และสามารถแมปข้อมูลจากในฐานข้อมูลเข้ากับแต่ละฟิลด์ของออบเจกต์ได้
- สามารถแก้ไขค่าได้ในหน่วยความจำเพื่อที่จะเปลี่ยนแปลงค่าข้อมูลจริง
- มีความคงทน สามารถเก็บลงในแหล่งบันทึกข้อมูลอีกครั้งได้เพื่อเป็นการอัปเดตข้อมูล

อินสแตนซ์ของ entity bean คือมุมมองที่มองลงไปยังข้อมูลที่อยู่ในหน่วยความจำเป็นอินสแตนซ์ของ entity bean class

ข้อมูลของ entity bean (entity bean data หรือ data instance) คือกลุ่มของข้อมูลในฐานข้อมูลในระดับกายภาพ เช่น ระเบียบของบัญชีธนาคาร

6.2.2 ส่วนประกอบของ Entity Bean

6.2.2.1 Entity Bean Class

คือคลาสในภาษาจาวาที่จำลองข้อมูลที่ persistent โดย entity bean class จะแมปเข้ากับหนึ่ง entity definition ใน database schema เช่น entity bean class หนึ่งสามารถแมปเข้าได้กับ definition ของตารางเชิงสัมพันธ์ ในกรณีนี้อินสแตนซ์ของ entity bean ของคลาสนั้นจะถูกแมปเข้ากับแถวหนึ่งในตาราง entity bean class สามารถมีเมธอดพื้นฐานเพื่อใช้จัดการหรือเข้าถึงข้อมูล เช่น เมธอดสำหรับลดยอดเงินในบัญชีธนาคาร เป็นต้น เช่นเดียวกับ session bean class EJB กำหนดเมธอดบังคับสำหรับ entity bean class ซึ่งคอนเทนเนอร์จะเรียกใช้เมธอดเหล่านี้โดยอัตโนมัติ

6.2.2.2 Remote Interface

เป็นอินเทอร์เฟซอ้างอิงไปยัง bean ซึ่งไคลเอนต์จะเข้ามาเรียกใช้ ภายใน remote interface จะต้องระบุถึงบิซิเนสเมธอดทั้งหมดที่ entity bean มีให้ใช้งาน ผู้พัฒนา EJB คอนเทนเนอร์จะให้เครื่องมือสำหรับอิมพลีเมนต์ remote interface มาให้ โดยส่วนอิมพลีเมนต์ของ remote interface ก็คือ EJB ออบเจกต์ ซึ่งทำหน้าที่เป็นชั้นกั้นกลางระหว่างไคลเอนต์กับ bean ไคลเอนต์จะร้องขอการใช้งาน bean มายัง EJB ออบเจกต์แทนที่จะร้องขอกับ bean โดยตรง

เนื่องจาก EJB ออบเจกต์เป็นส่วนหนึ่งของคอนเทนเนอร์ มันจะประกอบด้วยลอจิกในการดักจับการเรียกใช้เมธอด และจัดการอินสแตนซ์ของ bean ซึ่งเป็นหลักการเดียวกับ session bean

6.2.2.3 Home Interface

เป็นอินเทอร์เฟซที่ไคลเอนต์จะต้องเรียกใช้เพื่อสร้าง , ค้นหา , และทำลาย EJB ออบเจกต์ของ entity bean ภายใน home interface ต้องระบุเมธอดบอกถึงวิธีที่มีทั้งหมดในการสร้าง EJB ออบเจกต์ใหม่ขึ้นมา , การค้นหาและการทำลาย EJB ออบเจกต์เดิมที่มีอยู่แล้ว ผู้พัฒนา EJB คอนเทนเนอร์ จะให้เครื่องมือสำหรับ อิมพลีเมนต์ home interface นี้มาด้วย ส่วนอิมพลีเมนต์ของ home interface ก็คือ home object ซึ่งเปรียบเสมือนเป็นโรงงานที่ใช้ในการสร้าง , ค้นหา และทำลาย EJB ออบเจกต์ ในการค้นหา home object จะต้องเรียกใช้บริการ look up ของ JNDI เช่นเดียวกับหลักการของ session bean

6.2.2.4 Primary Key Class

เป็น unique identifier สำหรับ entity bean โดย primary key จะแยกความแตกต่างระหว่างแต่ละ entity bean เช่น ถ้ามี entity bean ประกอบด้วยบัญชีธนาคารทั้งหมดหนึ่งล้านบัญชี แต่ละบัญชีต้องมี unique ID (เช่น หมายเลขบัญชีธนาคาร) ซึ่งจะต้องไม่ซ้ำกันเลยในบัญชีทั้งหมด primary key เป็นออบเจกต์ที่สามารถประกอบด้วยหลาย attribute ได้ อาจเป็นข้อมูลที่จำเป็นในการแยกแยะอินสแตนซ์ของ entity bean ในกรณีที่มี entity bean เป็นตัวแทนของความสัมพันธ์ที่ซับซ้อน primary key อาจเป็นทั้งออบเจกต์เลขก็ได้ EJB ให้ความยืดหยุ่นในการระบุว่าอะไรจะเป็น unique ID โดยการรวม primary key class เข้ากับ entity bean มีกฎอยู่เพียงข้อเดียวว่า primary key class ต้องสามารถ serialize ได้ และเป็นไปตามกฎของ java object serialization

6.2.2.5 Deployment Descriptor

เก็บรายการคุณสมบัติ (properties) ต่าง ๆ ที่คอนเทนเนอร์ใช้ในการดีพลอย deployment descriptor ใช้แจ้งรายละเอียดเกี่ยวกับ bean ให้กับคอนเทนเนอร์ เมื่อจะนำ entity bean ไปใช้งาน ต้องรวมไฟล์เหล่านี้เข้าไว้ในไฟล์ ejb-jar รวมทั้ง manifest file เพื่อใช้ค้นหาตำแหน่งของ bean ในไฟล์ ejb-jar ด้วย

6.2.3 ลักษณะของ Entity Bean

■ Entity Bean มีช่วงชีวิตยาวนาน

session bean มีอายุเท่ากับ session ของไคลเอนต์เท่านั้น เมื่อไคลเอนต์จบการติดต่อก็สามารถทำลาย session bean ได้ทันที ส่วน entity bean จะมีช่วงชีวิตยาวนานเท่ากับคอนเทนเนอร์ โดยเริ่มและจบการทำงานพร้อมกับ คอนเทนเนอร์ เช่น ข้อมูลบัญชีธนาคาร

■ Entity Bean มีความทนต่อความผิดพลาด

session bean มีช่วงชีวิตที่สั้น เพียงแค่ session ของไคลเอนต์เท่านั้น และเมื่อมีเหตุการณ์ผิดพลาด เช่น JVM แครชเกิดขึ้น session bean จะถูกทำลายทันที แต่ entity bean เป็นส่วนหนึ่งของ persistent storage ดังนั้นการแครชของ JVM หรือฐานข้อมูล จะไม่มีผลกระทบต่อ entity bean เมื่อระบบกลับมาทำงานได้อีกครั้ง อินสแตนซ์ของ entity bean ก็สามารถสร้างขึ้นใหม่ได้ โดยการอ่านข้อมูลจากฐานข้อมูล และนำมาสร้างเป็นอินสแตนซ์ของ entity bean เพื่อเป็นตัวแทนของข้อมูลนั้นในหน่วยความจำได้

■ อินสแตนซ์ของ Entity Bean เป็นมุมมองไปยังฐานข้อมูล

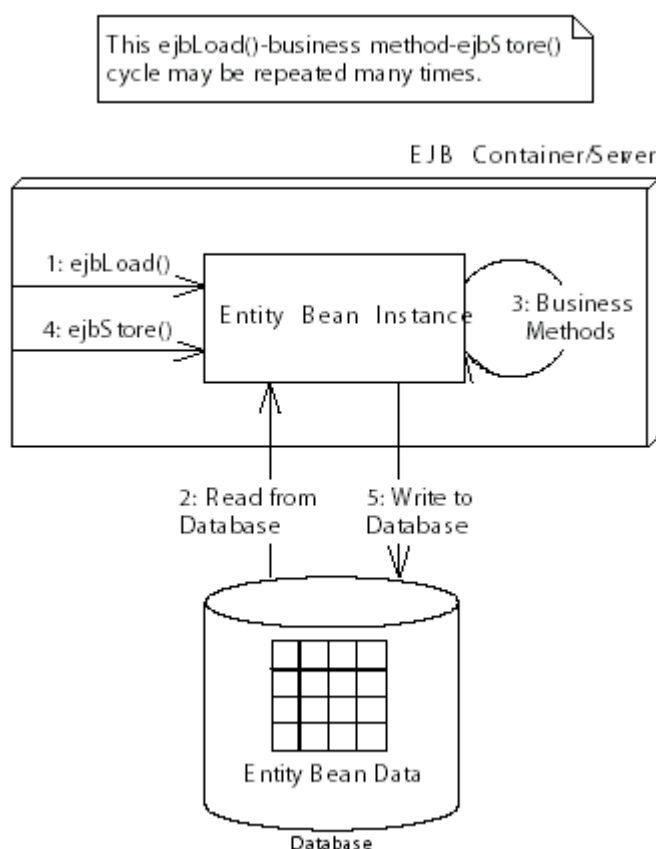
ในการโหลด entity bean เข้ามาเป็นอินสแตนซ์ในหน่วยความจำ จะต้องอ่านข้อมูลที่เก็บอยู่ในฐานข้อมูลแล้วนำมาจัดการใน java virtual machine โดยมองว่าออบเจกต์ที่อยู่ในหน่วยความจำและฐานข้อมูลนั้นเป็นสิ่งเดียวกัน หมายความว่า เมื่อแก้ไขอินสแตนซ์ของ entity bean ซึ่งอยู่ในหน่วยความจำ ข้อมูลในฐานข้อมูลจะต้องถูกเปลี่ยนแปลงด้วยโดยอัตโนมัติ entity bean ที่อยู่ในหน่วยความจำนั้น เสมือนเป็นมุมมองไปยังฐานข้อมูล

ในความเป็นจริงแล้ว ข้อมูลหนึ่ง ๆ จะมีอยู่สองชุดด้วยกัน คืออินสแตนซ์ของ entity bean ซึ่งอยู่ในหน่วยความจำ และตัว entity bean จริง ๆ ที่ถูกเก็บอยู่ในฐานข้อมูล ดังนั้นจึงต้องมีวิธีการส่งผ่านข้อมูลไปมาระหว่างจาวาออบเจกต์และฐานข้อมูล การส่งผ่านข้อมูลนี้สามารถทำได้โดยเมธอดที่ทุก entity bean class ต้องอิมพลีเมนต์ นั่นคือ ejbLoad() และ ejbStore()

ejbLoad() อ่านข้อมูลจาก persistent storage เข้าไปยังฟิลด์ของ entity bean ในหน่วยความจำ

ejbStore() บันทึกค่าในฟิลด์ปัจจุบันของอินสแตนซ์ของ bean ลงไปยังแหล่งเก็บข้อมูล เป็นการทำงานที่ตรงข้ามกับ ejbLoad()

เมธอดทั้งสองเป็น callback method ที่คอนเทนเนอร์จะเรียกใช้งานโดยอัตโนมัติ เป็นเมธอดการจัดการที่ EJB บังคับให้ต้องมี คอนเทนเนอร์จะพิจารณาเองว่า จะเรียกใช้งานเมธอดเหล่านี้เมื่อใด ขึ้นอยู่กับความสามารถของคอนเทนเนอร์ bean อาจถูกเรียกใช้งานเมธอดเหล่านี้เมื่อใดก็ได้ โดยคอนเทนเนอร์จะพิจารณาจากสถานะของทรานแซกชัน และ synchronize ข้อมูลให้โดยอัตโนมัติ โดยปกติแล้วลำดับจะเป็นดังรูปที่ 6-3



รูปที่ 6-3 การโหลดและการเก็บข้อมูลของ Entity Bean

แต่ในบางคอนเทนเนอร์สามารถกำหนดคุณสมบัติให้ entity bean เป็น read-only เท่านั้น ทำให้ไม่จำเป็นต้องเรียก ejbStore()

■ อินสแตนซ์ของ Entity Bean หลายตัวอาจแสดงข้อมูลเดียวกัน

เมื่อมีไคลเอนต์หลายตัวเข้ามาใช้งานข้อมูลต่าง ๆ ในเวลาเดียวกัน จำเป็นต้องออกแบบวิธีการเข้าถึงข้อมูลที่มีประสิทธิภาพสูงให้กับ entity bean โดย entity bean ไม่อนุญาตให้ไคลเอนต์หลายตัวใช้อินสแตนซ์ของ entity bean ตัวเดียวพร้อม ๆ กัน เนื่องจากอินสแตนซ์ของ bean ที่ทำเช่นนี้ได้จะต้องเขียนแบบ thread-safe ซึ่งมีความซับซ้อนและผิดพลาดได้ง่าย ตรงข้ามกับวัตถุประสงค์ของ EJB ที่ต้องการให้การพัฒนาแอปพลิเคชันทำได้ง่ายและรวดเร็ว ปัญหาอีกข้อหนึ่งคือ ในขณะที่มีหลาย ๆ เธรดทำงานพร้อมกันนั้น การควบคุมทรานแซกชันโดยระบบจัดการทรานแซกชันจะทำได้ยากมาก

จากปัญหาดังกล่าว EJB จึงอนุญาตให้มีเพียงเธรดเดียวเท่านั้น ที่สามารถทำงานอยู่ในอินสแตนซ์ของ bean ไม่ว่าจะเป็นอินสแตนซ์ของ session bean หรือ entity bean ก็ตาม

เพื่อเพิ่มประสิทธิภาพในการทำงาน คอนเทนเนอร์สามารถสร้างอินสแตนซ์ของ entity bean หนึ่ง ๆ ขึ้นมาได้หลายชุด ทำให้ไคลเอนต์หลายตัวสามารถทำงานพร้อมกันได้ โดยแต่ละไคลเอนต์จะทำงานกับอินสแตนซ์หนึ่งตัว โดยที่อินสแตนซ์ของ entity bean เหล่านั้นสามารถแทนข้อมูลชุดเดียวกัน

การมีอินสแตนซ์ของ bean หลาย ๆ ชุดแทนข้อมูลชุดเดียวกันนั้น ทำให้เกิดปัญหาใหม่ตามมา คือ ปัญหาเรื่องความถูกต้องของข้อมูล เมื่อไคลเอนต์หลายตัวพยายามจะแก้ไขข้อมูลพร้อม ๆ กัน และอาจทำให้มีไคลเอนต์ที่ได้ข้อมูลผิดพลาดไป เพื่อแก้ปัญหานี้คอนเทนเนอร์จะทำการ synchronize ข้อมูลของ bean กับข้อมูลที่อยู่ในฐานข้อมูลด้วยการเรียกใช้งานเมธอด ejbLoad() และ ejbStore()

ความถี่ของการ synchronize ข้อมูล คอนเทนเนอร์จะพิจารณาจากทรานแซกชัน ทรานแซกชันทำให้ไคลเอนต์รู้สึกเหมือนกับว่าทำงานกับข้อมูลนั้นเพียงคนเดียว ทั้งที่ความเป็นจริงมีไคลเอนต์อีกหลายตัวกำลังทำงานกับข้อมูลชุดเดียวกันนั้นอยู่

■ อินสแตนซ์ของ Entity Bean สามารถพูลได้

เพื่อประหยัดเวลาในการสร้างและทำลายอินสแตนซ์ของ bean ทุกครั้งที่มีไคลเอนต์ติดต่อเข้ามา และเพื่อเพิ่มประสิทธิภาพในการทำงาน อินสแตนซ์ของ entity bean เป็นออบเจกต์ที่สามารถนำกลับมาใช้ใหม่และพูลได้ คอนเทนเนอร์สามารถพูลและนำอินสแตนซ์ของ entity bean กลับไปใช้ใหม่เพื่อแทนอินสแตนซ์ของข้อมูลประเภทเดียวกันได้ เช่น คอนเทนเนอร์สามารถใช้อินสแตนซ์ของ entity bean บัญชีธนาคาร เพื่อแทนบัญชีหนึ่ง ๆ ในระบบบัญชีธนาคาร เมื่อใช้งานอินสแตนซ์นี้เสร็จแล้วก็สามารถนำไปให้บริการกับ ไคลเอนต์ตัวอื่น ๆ ได้ต่อไปและอาจแทนข้อมูลคนละชุดกับของเดิม คอนเทนเนอร์จัดการโดยการกำหนด อินสแตนซ์ของ entity bean ให้กับ EJB ออบเจกต์ที่แตกต่างกันออกไปในแต่ละไคลเอนต์

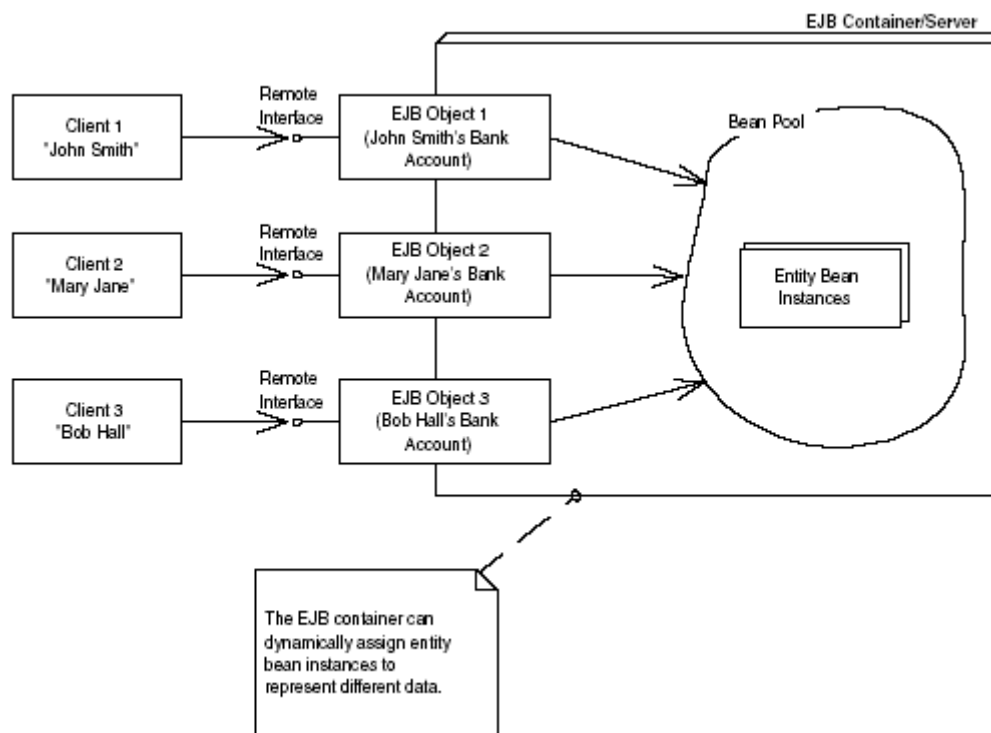
นอกจากจะช่วยประหยัดเวลาในการสร้างและทำลายอินสแตนซ์ของ bean โดยไม่จำเป็นแล้ว วิธีการนี้ยังช่วยลดทรัพยากรที่ระบบจำเป็นต้องใช้อีกด้วย ดังแสดงในรูปที่ 6-4

ในทำนองเดียวกับ stateful session bean การกำหนดอินสแตนซ์ของ entity bean ให้กับอีก EJB ออบเจกต์หนึ่งนั้นมีความซับซ้อน เมื่อ entity bean ถูกกำหนดให้กับ EJB ออบเจกต์ใด ๆ มันอาจมีการใช้งานทรัพยากร เช่น socket connection เป็นต้น แต่ในเวลาที่อยู่ในพูล ไม่มีความจำเป็นที่จะต้องใช้ซ้ำอีก เกิด ดังนั้นเพื่อให้ bean สามารถปล่อย และกลับมาใช้งานทรัพยากรต่าง ๆ entity bean จะต้องอิมพลิเมนต์ callback method ทั้งสองนี้

ejbActivate() เป็นเมธอดที่คอนเทนเนอร์เรียกใช้งานเมื่อนำอินสแตนซ์ของ bean ออกจากพูล กระบวนการนี้เรียกว่า activation บอกให้รู้ว่า คอนเทนเนอร์กำลังกำหนด bean ให้กับ EJB ออบเจกต์และ primary key หนึ่ง เมธอด ejbActivate() ควรจะร้องขอทรัพยากรที่ต้องใช้งานเมื่อถูกกำหนดให้กับ EJB ออบเจกต์แล้ว เช่น ซ็อกเก็ต หลักการนี้เป็นหลักการเกี่ยวกับการทำ activation ของ stateful session bean

ejbPassivate() เป็นเมธอดที่คอนเทนเนอร์เรียกใช้งานเมื่อนำอินสแตนซ์ของ bean เก็บเข้าไปในพูล กระบวนการนี้เรียกว่า passivation เพื่อบอกให้รู้ว่า คอนเทนเนอร์กำลังถอน bean ออกจาก EJB ออบเจกต์

เมธอด `ejbPassivate()` ควรทำการปล่อยทรัพยากรที่ถือครองอยู่ทั้งหมด เช่นชื่อเก็ตซึ่ง bean ได้ถือครองไว้ขณะ activation หลักการนี้เป็นหลักการเดียวกับการทำ passivation ของ stateful session bean

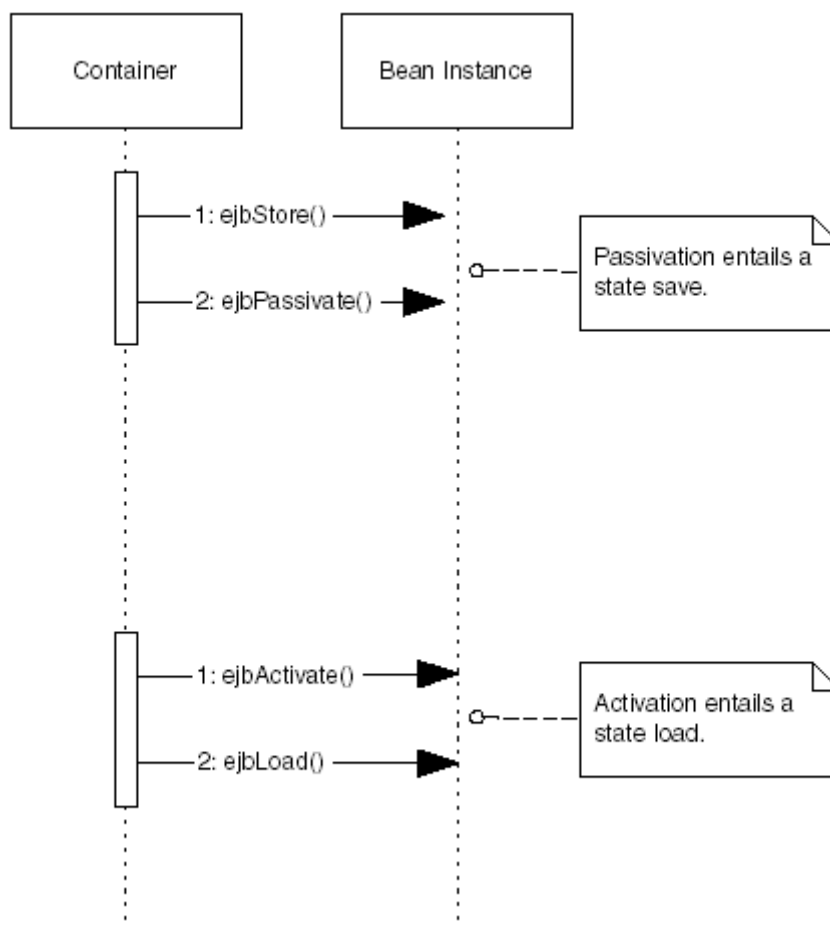


รูปที่ 6-4 การพลูอินสแทนซ์ของ Entity Bean โดยคอนเทนเนอร์

เมื่ออินสแทนซ์ของ entity bean ถูก passivate ไม่เพียงแต่จะต้องปล่อยทรัพยากรที่ถือครองเท่านั้น แต่ต้องบันทึกสถานะของมันลงไปในแหล่งเก็บข้อมูลด้วย เพื่อให้ข้อมูลในแหล่งเก็บข้อมูลถูกอัปเดตให้ตรงกับสถานะล่าสุดของอินสแทนซ์ของ entity bean ในการบันทึกฟิลด์ของอินสแทนซ์ลงในฐานข้อมูล คอนเทนเนอร์จะเรียกใช้งานเมธอด `ejbStore()` ก่อนทำ passivation ทำนองเดียวกัน เมื่ออินสแทนซ์ของ entity bean ถูก activate ไม่เพียงแต่จะต้องร้องขอการใช้งานทรัพยากรที่จำเป็น แต่จะต้องโหลดข้อมูลล่าสุดขึ้นมาจากฐานข้อมูลด้วย ในการโหลดข้อมูลล่าสุดจากฐานข้อมูลนี้ คอนเทนเนอร์จะเรียกใช้เมธอด `ejbLoad()` หลังจากทำ activation วิธีการนี้แสดงอยู่ในรูปที่ 6-5

entity bean มีความคล้ายคลึงกับ stateful session bean ตรงที่สามารถทำ activation และ passivation ได้ แต่ข้อแตกต่างที่สำคัญคือ entity bean มีเมธอด `ejbLoad()` เพื่อโหลดค่าสถานะขณะทำ activation และ เมธอด `ejbStore()` เพื่อบันทึกสถานะขณะทำ passivation entity bean เป็นออบเจกต์ที่มีความซับซ้อนกว่า stateful session bean มาก และต้องการวิธีการที่ซับซ้อนมากกว่า object serialization อินสแทนซ์ของ entity bean อาจเก็บข้อมูลของตัวเองลงในฐานข้อมูลแบบออบเจกต์โดยใช้ API ของฐานข้อมูลแบบออบเจกต์ ทำให้ entity bean ต้องการเมธอด `ejbLoad()` และ `ejbStore()` เพื่อจัดการเกี่ยวกับการเก็บค่า

สถานะ ขณะที่ stateful session bean ไม่ต้องการเมธอดเหล่านี้ แต่จะใช้วิธีการ object serialization ในการบันทึกสถานะของ stateful session bean



รูปที่ 6-5 การทำ passivation และ activation ของ Entity Bean

■ Entity Bean สามารถทำ persistent ได้สองวิธี

entity bean เป็น persistent object ที่รู้ถึงวิธีการแปลงตัวมันเองลงในแหล่งบันทึกข้อมูล เช่น ฐานข้อมูลเชิงสัมพันธ์ โดยใช้หลักการ O/R mapping หรือฐานข้อมูลเชิงวัตถุ มีวิธีการทำ persistent 2 วิธี

1) Bean-Managed Persistent

วิธีนี้ bean เป็นผู้จัดการเกี่ยวกับการติดต่อกับแหล่งบันทึกข้อมูลเอง โดยอิมพลีเมนต์การเรียกใช้ฐานข้อมูลลงไป bean เช่น เมื่อทำงานกับฐานข้อมูลเชิงสัมพันธ์ entity bean สามารถทำคำสั่ง SQL INSERT ผ่านทาง JDBC เพื่อบันทึกข้อมูลลงไปฐานข้อมูลเชิงสัมพันธ์ หรืออาจทำคำสั่ง SQL DELETE ผ่านทาง JDBC เพื่อลบข้อมูลออกจากฐานข้อมูล

2) Container-Managed Persistent

วิธีนี้ให้คอนเทนเนอร์เป็นผู้จัดการเกี่ยวกับ persistent ให้ ทำให้สามารถตัดลอจิกที่เกี่ยวข้องกับการ persistent ข้อมูลออกจาก bean ได้ ไม่จำเป็นต้องเขียนโค้ดลงไป bean คลาส เพียงแต่บอก EJB คอน

แทนเนอร์ว่า 필ด์ใดเป็น persistent 필ด์ โดยระบุไว้ใน deployment descriptor เมื่อคอนเทนเนอร์ทราบแล้วว่า 필ด์ใดบ้างที่ต้อง persistent คอนเทนเนอร์จะจัดการลอจิกเกี่ยวกับการเข้าถึงข้อมูลให้อัตโนมัติ

การจัดการเกี่ยวกับ persistent โดยอัตโนมัติ เป็นหน้าที่ของผู้ดีพลอยที่จะระบุว่า entity bean จะแมปไปยังแหล่งเก็บข้อมูลอย่างไร เช่น 필ด์ของจาวาออบเจกต์ชื่อ customerID จะต้องแมปเข้ากับคอลัมน์ชื่อ id ในตาราง customer ของฐานข้อมูล เป็นต้น สิ่งนี้เป็นจุดเด่นที่สำคัญของ entity bean เพราะสามารถเขียนออบเจกต์ของข้อมูลที่ไม่ขึ้นกับแหล่งบันทึกข้อมูลได้ และสามารถใช้ได้ในสภาพแวดล้อมที่หลากหลาย

■ Entity Bean สามารถสร้าง, ทำลาย หรือค้นหาได้

การกำหนดค่าเริ่มต้น และการทำลาย entity bean แตกต่างออกไปจาก session bean เล็กน้อย entity bean เป็นมุมมองลงไปยังฐานข้อมูล และมองอินสแตนซ์ของ entity bean และข้อมูลที่อยู่ในฐานข้อมูลเป็นสิ่งเดียวกัน เพราะข้อมูลจะถูก synchronize กันอยู่เสมอ การสร้างและกำหนดค่าเริ่มต้นให้กับอินสแตนซ์ของ entity bean จึงเป็นการสร้างและกำหนดค่าเริ่มต้นให้กับข้อมูลในฐานข้อมูลด้วย ดังนั้นเมื่อ entity bean ถูกสร้างและกำหนดค่าเริ่มต้นในหน่วยความจำด้วยเมธอด ejbCreate() เมธอดนี้จะสร้างข้อมูลใหม่ในฐานข้อมูลซึ่งสัมพันธ์กับอินสแตนซ์ในหน่วยความจำ ในทำนองเดียวกัน เมื่อเมธอด ejbRemove() ถูกเรียกใช้งาน ข้อมูลในฐานข้อมูลที่สัมพันธ์กันก็จะถูกลบทิ้ง ถ้าเป็นแบบ container-managed persistence คอนเทนเนอร์จะแก้ไขข้อมูลในฐานข้อมูลให้อัตโนมัติ จึงไม่ต้องอิมพลิเมนต์เมธอดทั้งสองนี้

ข้อมูลของ entity bean สามารถระบุเฉพาะตัวได้ จึงสามารถใช้การค้นหา entity bean แทนที่จะต้องสร้างขึ้นมาใหม่ การค้นหา entity bean เหมือนกับการใช้คำสั่ง SQL SELECT การค้นหา entity bean ก็คือการค้นหาข้อมูลในแหล่งบันทึกข้อมูล แตกต่างจาก session bean ที่ไม่สามารถค้นหาได้ เพราะ session bean นั้นไม่ได้เป็นออบเจกต์ที่ถาวร

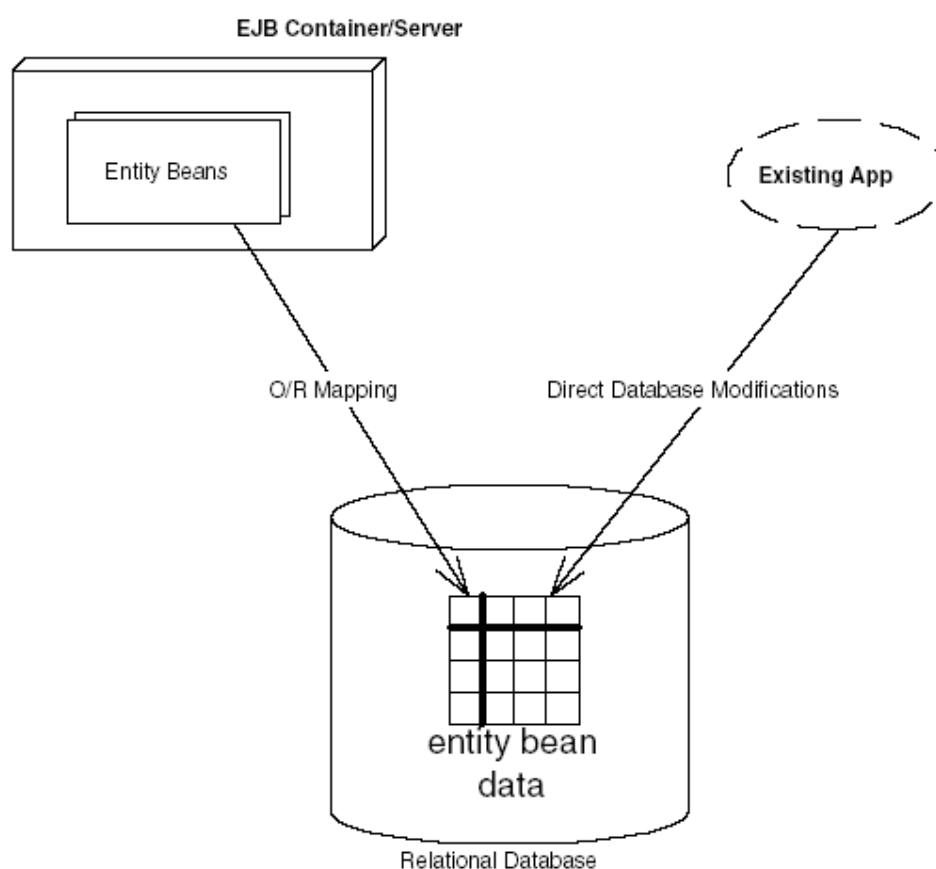
วิธีค้นหา entity bean มีได้หลายวิธีด้วยกัน โดยจะระบุวิธีการเหล่านี้เป็นเมธอดไว้ใน home interface ของ entity bean เรียกว่า finder method นอกเหนือจากเมธอดที่ใช้ในการสร้างและทำลาย entity bean จุดนี้เป็นจุดที่ home interface ของ session bean และ entity bean แตกต่างกัน เพราะใน home interface ของ session bean ไม่มี finder method

■ Entity Bean สามารถใช้เป็นตัวแทน Legacy Data และ Legacy Systems ได้

entity bean สามารถเกิดก่อนที่จะมีการนำ EJB มาใช้ได้ เพราะ entity bean ก็คือข้อมูล ดังนั้นสามารถนำระบบฐานข้อมูลเก่า (legacy database) หรือระบบเก่า (legacy system) และใช้ entity bean เป็นตัวแทนข้อมูลเหล่านี้ได้

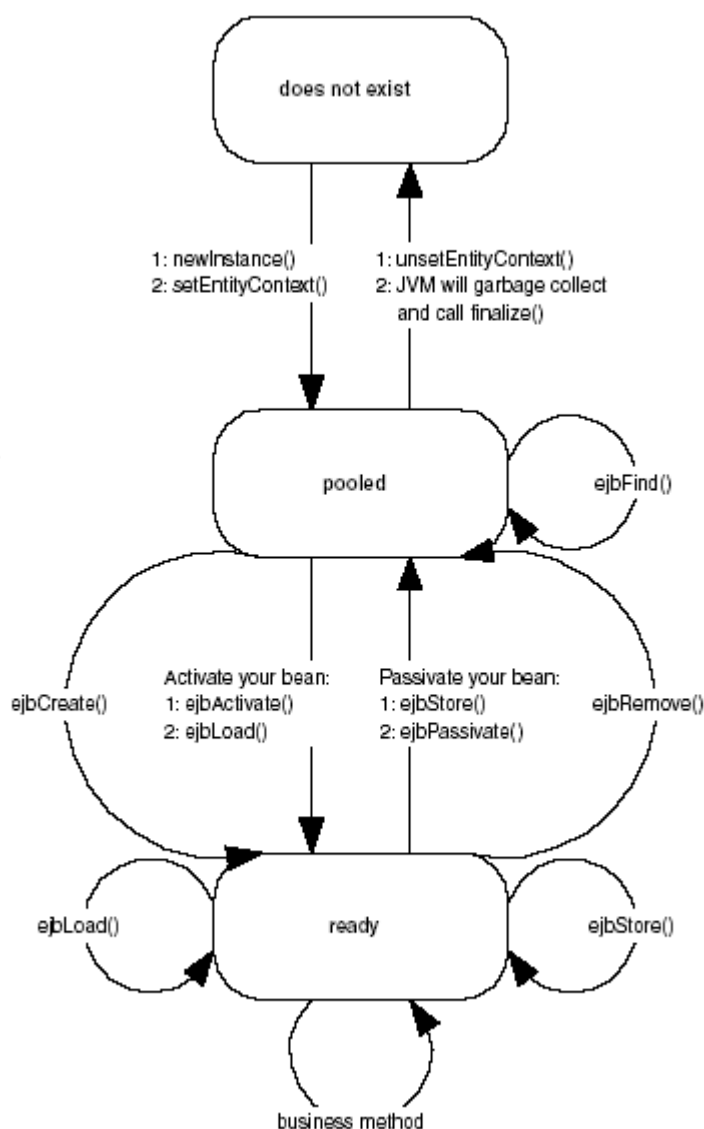
■ **Entity Bean สามารถแก้ไขได้โดยไม่ต้องผ่าน EJB**

ปกติ การสร้าง , ทำลาย และค้นหาข้อมูลของ entity bean ทำโดยใช้ home interface แต่สามารถติดต่อกับ entity bean ได้อีกริธีหนึ่ง คือการแก้ไขฐานข้อมูลซึ่งบันทึกข้อมูลนั้นอยู่โดยตรง เช่น ถ้าอินสแตนซ์ของ bean ถูกแมปเข้ากับฐานข้อมูลเชิงสัมพันธ์ จึงสามารถลบแถวออกจากฐานข้อมูลซึ่งสัมพันธ์กับอินสแตนซ์ของ entity bean ได้โดยตรง (รูป 6-6) , สามารถสร้างข้อมูล entity bean ใหม่ และแก้ไขข้อมูลที่มีอยู่แล้ว โดยการทำงานกับฐานข้อมูลโดยตรง การทำเช่นนี้อาจมีความจำเป็นหากระบบงานส่วนหนึ่งเป็นระบบงานเก่าที่แก้ไขข้อมูลโดยการติดต่อกับฐานข้อมูลโดยตรง



รูปที่ 6-6 การแก้ไขฐานข้อมูลที่สัมพันธ์กับ Entity Bean โดยตรง

วงจรชีวิตของ entity bean ดังรูปที่ 6-7 ซึ่งจะเห็นว่ามีเมธอดที่เพิ่มเติมขึ้นมาก็คือ ejbFind() , ejbLoad() และ ejbCreate()



รูปที่ 6-7 วงจรชีวิตของ Entity Bean

6.2.4 ตัวอย่างโค้ด Entity Bean

ในการเขียน entity bean class จะต้องอิมพลิเมนต์อินเทอร์เฟซ javax.ejb.EntityBean อินเทอร์เฟซจะกำหนดเมธอดที่ entity bean class จะต้องอิมพลิเมนต์ อินเทอร์เฟซ javax.ejb.EnterpriseBean ไม่ได้กำหนดเมธอดใด ๆ ไว้ ดังนี้

```
public interface javax.ejb.EnterpriseBean implements {
}
```

อินเทอร์เฟซ javax.ejb.EntityBean จะกำหนด callback method ที่ bean ต้องอิมพลีเมนต์ ดังนี้

```
public interface javax.ejb.EntityBean implements javax.ejb.EnterpriseBean{

    public abstract void setEntityContext(javax.ejb.EntityContext);

    public abstract void unsetEntityContext();

    public abstract void ejbRemove();

    public abstract void ejbActivate();

    public abstract void ejbPassivate();

    public abstract void ejbLoad();

    public abstract void ejbStore();

}
```

นอกเหนือจากเมธอดที่เห็น bean จำเป็นที่จะต้องกำหนดเมธอด ejbCreate() ที่ใช้ในการสร้างข้อมูลของ entity bean ใหม่และ ejbFind() ที่ใช้ในการค้นหาข้อมูลของ bean ที่มีอยู่

- ejbCreate()

ejbCreate() ใช้ในการสร้างข้อมูลใหม่ในฐานข้อมูล เมธอดนี้จะมีหรือไม่มีก็ได้ หากไม่ต้องการสร้างข้อมูลใหม่ขึ้นมาก็ไม่จำเป็นต้องมีเมธอดนี้ ไม่เหมือนกับ session bean ที่จะบังคับให้มีอย่างน้อย 1 เมธอด พารามิเตอร์ของเมธอดนี้จะได้หลากหลายขึ้นอยู่กับข้อมูลในฐานข้อมูลที่ต้องการสร้าง และ ejbCreate() จะต้องมีเมธอด create() ที่มีพารามิเตอร์ตรงกับ ejbCreate() นั้นใน home interface ตัวอย่างเช่น bank account entity bean class ที่ชื่อว่า AccountBean กับ remote interface ที่ชื่อ Account , home interface ชื่อ AccountHome และ primary key class ชื่อ AccountPK โดยให้เมธอด ejbCreate() ใน AccountBean เป็นดังนี้

```
public AccountPK ejbCreate(String accountId,String owner) ...
```

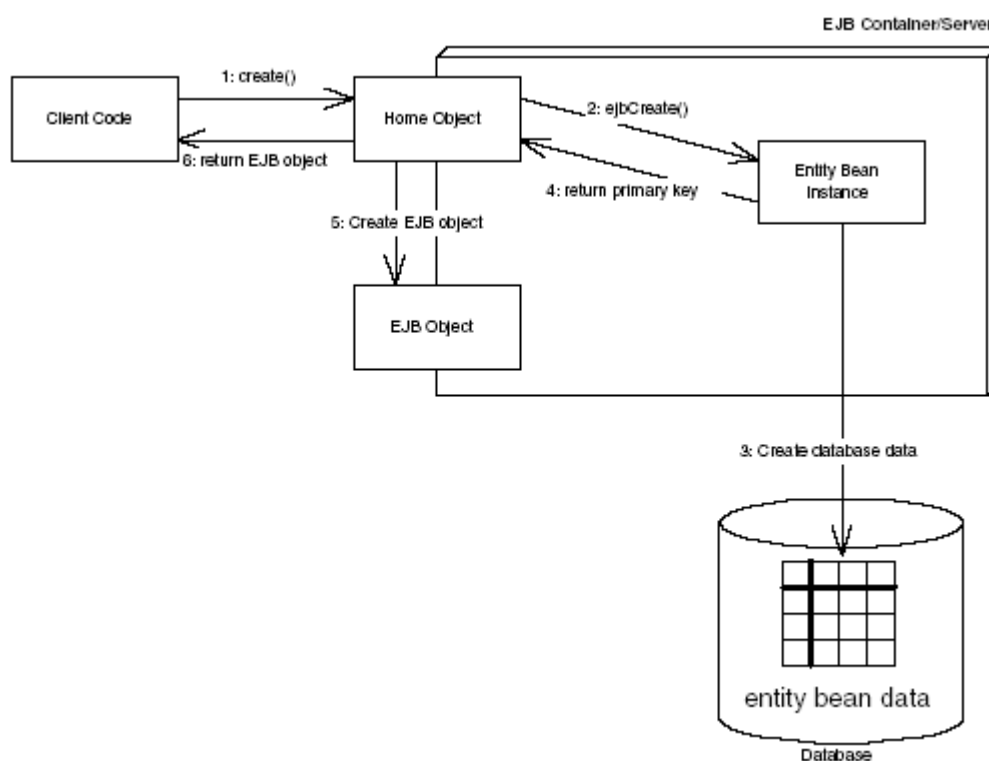
จะต้องมีเมธอด create() ใน home interface ดังนี้

```
public Account create(String accountId,String owner) ...
```

สังเกตข้อแตกต่างระหว่างค่าที่คืนมาใน 2 เมธอดนี้ จะพบว่าอินสแตนซ์ของ bean จะคืนค่า primary key ขณะที่ home object จะคืนค่า EJB object ที่เป็นเช่นนั้นเนื่องจากคอนเทนเนอร์จะคืนค่า primary key ให้กับ home object และ home object จะนำไปสร้าง EJB object แล้วคืนค่า EJB object ให้กับไคลเอนต์ ดังรูปที่ 6-8

- ejbFind()

finder method ใช้ในการหา entity bean ที่อยู่ใน storage โดยสามารถมีได้หลายเมธอดดังนี้



รูปที่ 6-8 ความสัมพันธ์ระหว่าง `ejbCreate()` และ `create()`

```
public AccountPK ejbFindByPrimarykey(AccountPK key)
public Enumeration ejbFindAllProducts()
public Enumeration ejbFindBigAccounts(int minimum)
public OrderPK ejbFindMostRecentOrder()
```

finder method มีกฎคือ จะต้องขึ้นด้วย `ejbFind` , ต้องมีอย่างน้อย 1 เมธอดคือ `ejbFindByPrimarykey` , ค่าที่คืนจะต้องเป็น primary key ของ entity bean หรือ enumeration ของ primary key และเหมือนกับ `ejbCreate()` ก็ต้องมีเมธอดที่ตอบสนองคู่กับ `ejbFind` ดังตัวอย่างนี้ finder method ใน `AccountBean` ดังนี้

```
public Enumeration ejbFindAllProducts() ...
```

```
public AccountPK ejbFindBigAccounts(int minimum) ...
```

แล้วจะต้องมีเมธอดใน home interface ดังนี้

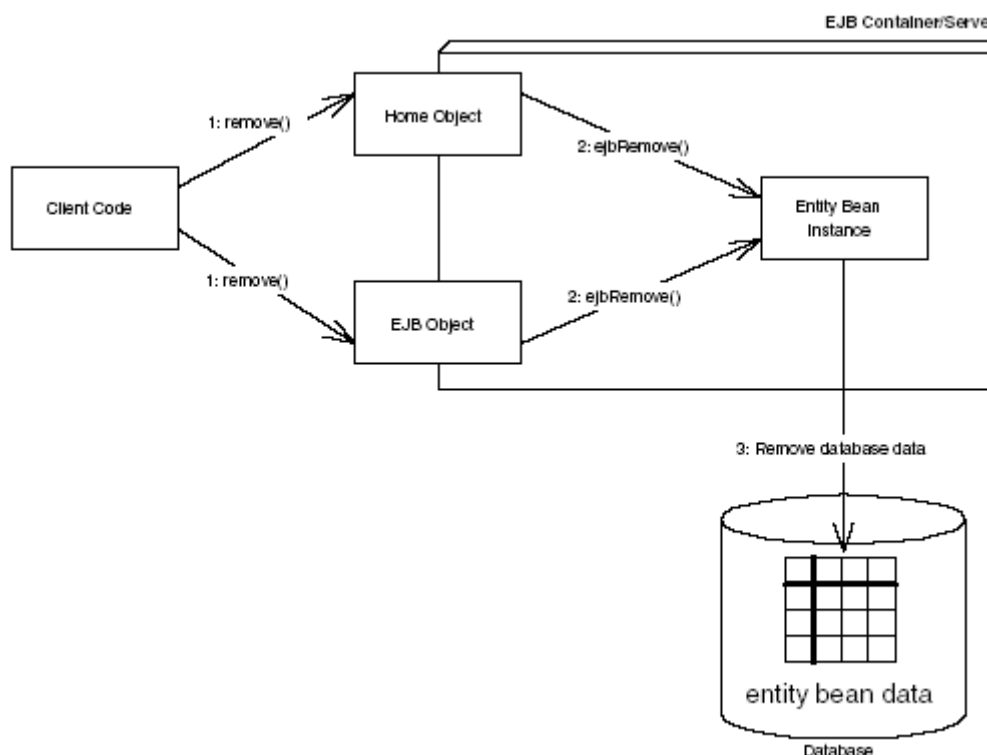
```
public Enumeration findAllProducts() ...
```

```
public Account ejbFindBigAccounts(int minimum) ...
```

สังเกตว่าในกรณีที่ไม่ใช่ enumeration ค่าที่คืนจะต่างกันด้วยเหตุผลเดียวกับ `ejbCreate()` และก็มีกฎพิเศษอีกอย่างหนึ่งคือในกรณีที่มันเป็น container-managed persistence EJB คอนเทนเนอร์จะอิมพลีเมนต์ finder method ใน EJB object ให้โดยอัตโนมัติ

- ejbRemove()

ไคลเอ็นต์จะใช้เมธอด remove() ในการลบข้อมูลของ entity bean ในฐานข้อมูล โดยเรียกเมธอด remove() กับ EJB object หรือ home object แล้วคอนเทนเนอร์จะเรียกเมธอด ejbRemove() กับ bean ดังรูปที่ 6-9 เมธอดนี้จะไม่ทำลายอินสแตนซ์ของ entity bean แต่จะลบข้อมูลในฐานข้อมูลเท่านั้น



รูปที่ 6-9 การลบข้อมูลของ Entity Bean

6.3 Bean-Managed Persistent Entity Bean

entity bean ทั้ง 2 ชนิดจะต้องอิมพลีเมนต์อินเทอร์เฟซ javax.ejb.EntityBean โดยอินเทอร์เฟซจะกำหนด callback method ที่คอนเทนเนอร์จะเรียกใช้กับ bean เมธอดที่ต้องอิมพลีเมนต์นั้นขึ้นอยู่กับชนิดของ entity bean ตารางที่ 6-1 จะอธิบายถึงเมธอดสำหรับ bean-managed persistent

เมธอด	คำอธิบาย
setEntityContext()	เมื่อคอนเทนเนอร์สร้างอินสแตนซ์ของ bean แล้วจะเรียกเมธอดนี้
ejbFind<...>(<...>)	finder method ใช้ในการค้นหาอินสแตนซ์ของ entity bean ที่มีข้อมูลที่ต้องการ โดย bean จะต้องอิมพลีเมนต์อย่างน้อยหนึ่งเมธอดคือ ejbFindByPrimaryKey()
EjbCreate(<...>)	เมื่อไคลเอ็นต์เรียกเมธอด create() กับ home interface คอนเทนเนอร์จะเรียกเมธอด ejbCreate() กับอินสแตนซ์ของ bean ที่พูลอยู่ ejbCreate() จะทำหน้าที่ในการสร้างข้อมูลใหม่ในฐานข้อมูลให้
ejbPostCreate(<...>)	bean class จะต้องกำหนดเมธอด ejbPostCreate() ให้เข้าคู่กับ ejbCreate() คือต้อง

	มีพารามิเตอร์เหมือนกัน คอนเทนเนอร์จะเรียกเมธอดนี้หลังจากเรียก <code>ejbCreate()</code>
<code>ejbActivate()</code>	เมื่อไคลเอนต์ต้องการเรียกบิซิเนสเมธอดกับ EJB object แต่ไม่มีอินสแตนซ์ของ entity bean นั้นที่ bind กับ EJB object คอนเทนเนอร์จะนำ bean จากพูลและเปลี่ยนสถานะเป็น ready โดยเมธอดนี้จะไม่ถูกเรียกระหว่างทรานแซกชัน
<code>ejbLoad()</code>	คอนเทนเนอร์จะเรียกเมธอดนี้เพื่อโหลดข้อมูลจากฐานข้อมูลไปยังอินสแตนซ์ของ bean
<code>ejbStore()</code>	คอนเทนเนอร์จะเรียกเมธอดนี้เพื่ออัปเดตข้อมูลจากอินสแตนซ์ของ bean ลงฐานข้อมูล
<code>ejbPassivate()</code>	คอนเทนเนอร์จะเรียกเมธอดนี้เมื่อนำ entity bean เก็บลงในพูล
<code>ejbRemove()</code>	จะลบข้อมูลในฐานข้อมูล
<code>unsetEntityContext()</code>	เมธอดนี้ใช้ในการลบ bean ออกจากเอนไวรอนเมนต์ คอนเทนเนอร์จะเรียกเมธอดนี้ก่อนที่อินสแตนซ์ของ entity bean จะถูกทำลาย

ตารางที่ 6-1 คำอธิบายเมธอดใน *Bean-Managed Persistent Entity Bean*

6.3.1 ตัวอย่างโค้ด Bean-Managed Persistent Entity Bean

ตัวอย่าง bean นี้คือ bean ที่เก็บข้อมูลของบัญชีธนาคาร สคริปต์ที่ใช้สร้างเทเบิลดังนี้

```
create table accounts (id varchar(64), ownername varchar(64), balance numeric(18));
```

6.3.1.1 Remote Interface

remote interface ขยายมาจาก `javax.ejb.EJBObject` เหมือน remote interface ของ bean ชนิดอื่น ๆ สังกัดที่เมธอด `withdraw(double amt)` จะพบว่ามี exception แบบกำหนดเองคือ `AccountException` ที่จะโยนออกมาเมื่อจะถอนเงินมากกว่าจำนวนเงินที่มีอยู่ โค้ดดังนี้

Account.java

```
package olala;

import javax.ejb.EJBObject;
import java.rmi.RemoteException;

public interface Account extends EJBObject {

    public double getBalance() throws RemoteException;

    public void deposit(double amt) throws RemoteException;

    public void withdraw(double amt) throws RemoteException, AccountException;

    public String getId() throws RemoteException;

    public void setId(String id) throws RemoteException;

    public String getOwnername() throws RemoteException;
```

```

public void setOwername(String owername)    throws RemoteException;
}

```

6.3.1.2 Home Interface

โค้ดดังนี้

AccountHome.java

```

package olala;

import javax.ejb.EJBHome;
import javax.ejb.FinderException;
import java.rmi.RemoteException;
import javax.ejb.CreateException;
import java.util.Enumuration;

public interface AccountHome extends EJBHome
{
    public Account findByPrimaryKey(AccountPK key)
        throws FinderException, RemoteException;
    public Account create(String id, String owername)
        throws CreateException, RemoteException;
    public Account create(String id, String owername, double balance)
        throws CreateException, RemoteException;
    public Enumuration findByOwnerName(String name)
        throws FinderException, RemoteException;
}

```

สังเกตว่าเมธอด create() จะกำหนดให้มีการโยน javax.ejb.CreateExeption ออกมาได้ และเมธอด find() จะกำหนดให้มีการโยน javax.ejb.FinderException ออกมาได้

6.3.1.3 primary key class

primary key class จะมีหรือไม่ก็ได้ ถ้า primary key มี attribute เดียว (ตัวอย่างนี้คือ id) ตัวอย่างนี้เป็นตัวอย่างที่ใช้ primary key class ถ้าไม่ใช้ในส่วนของ descriptor แทนที่จะระบุคลาสของ primary key เป็น AccountPK ก็ระบุเป็น java.lang.String แทน เนื่องค่าตัวแปร id เป็นสตริง โค้ดดังนี้

AccountPK.java

```

package olala;

import java.io.Serializable;

```

```

public class AccountPK implements Serializable{

    public String id;

    public int hashCode() { return (id.hashCode()); }

    public boolean equals(Object that)  {

        if (!(that instanceof AccountPK))

            return false;

        AccountPK tmp = (AccountPK)that;

        return (this.id.equals(tmp.id));

    }

    public AccountPK() { }

    public AccountPK(String id) {

        this.id=id;

    }

    public String toString(){

        return id;

    }

}

```

6.3.1.4 Enterprise Bean Class

bean-managed persistent จะต้องแมปกับฐานข้อมูลโดยใช้ JDBC จึงทำให้โค้ดมีขนาดใหญ่และยุ่งยากในการเขียน โดยตัวอย่างโค้ดนี้จะรับ JDBC connection โดยใช้ datasource ซึ่งจะต้องกำหนดโดยดึงจาก resource reference ที่ระบุในไฟล์ XML ทำให้สามารถเปลี่ยนแปลง datasource ได้โดยไม่ต้องยุ่งเกี่ยวกับโค้ด ตัวอย่างนี้ใช้ชื่อ resource reference ว่า jdbc/OlalaDB และค่าของมันเป็นชื่อ JNDI ที่ชื่อ OlalaDB โดยได้กำหนดให้คอนเทนเนอร์ bind datasource กับ JNDI ไว้ วิธี bind datasource อยู่ในภาคผนวก ก โค้ดของ bean ดังนี้

AccountEJB.java

```

package olala;

import javax.ejb.*;
import java.sql.*;
import javax.sql.DataSource;
import javax.naming.*;
import java.util.*;

public class AccountEJB implements EntityBean{

```

```

private String id;

private String ownername;

private double balance;

private EntityContext context;

public AccountEJB() { }

public void setId(String id){ this.id = id; }

public String getOwnername(){ return(ownername); }

public void setOwnername(String ownername) { this.ownername = ownername; }

public double getBalance(){ return(balance); }

public void setEntityContext(EntityContext ec){ context = ec; }

public void unsetEntityContext(){ this.context = null;}

public void ejbActivate() { }

public void ejbPassivate() { }

private Connection getConnection() throws EJBException {
    final String DATASOURCE = "java:comp/env/jdbc/OlalaDB";
    try {
        InitialContext ic = new InitialContext();
        DataSource ds = (DataSource)ic.lookup(DATASOURCE);
        return ds.getConnection();
    }
    catch (Exception ex) {
        throw new EJBException(ex.getMessage());
    }
}

private void readData(AccountPK key)    throws FinderException, EJBException {
    Exception ex = null;
    Connection con = null;
    PreparedStatement ps = null;
    try {
        con = getConnection();
        ps = con.prepareStatement("select ownername,balance from olalaDB.dbo.accounts where id = ?");
        ps.setString(1, key.id);
        ResultSet rs = ps.executeQuery();
    }
}

```

```

        if (!rs.next())
            ex = new FinderException("Record not found");
        else {
            id = key.id;
            ownername = rs.getString(1);
            balance = rs.getDouble(2);
        }
        rs.close();
    }
    catch (EJBException ee) {
        ex = ee;
    }
    catch (SQLException se) {
        ex = new EJBException(se.getMessage());
    }
    try {
        ps.close();
        con.close();
    }
    catch (Exception e) { }
    if (ex != null) {
        if (ex instanceof EJBException)
            throw (EJBException)ex;
        if (ex instanceof FinderException)
            throw (FinderException)ex;
    }
}

public AccountPK ejbFindByPrimaryKey(AccountPK key) throws FinderException {
    try {
        readData(key);
        return key;
    }
    catch (EJBException ee) {

```

```

        throw new FinderException(ee.getMessage());
    }
}

public void ejbLoad() {
    AccountPK key = (AccountPK)context.getPrimaryKey();
    try {
        readData(key);
    }
    catch(Exception ex) {
        throw new RuntimeException(ex.getMessage());
    }
}

public void ejbStore() {
    RuntimeException ex = null;
    Connection con = null;
    PreparedStatement ps = null;
    try {
        con = getConnection();
        AccountPK key = (AccountPK)context.getPrimaryKey();
        ps = con.prepareStatement("update olalaDB.dbo.accounts set ownername = ?,balance = ? where id
= ?");
        if (ownername != null)
            ps.setString(1, ownername);
        else
            ps.setNull(1, java.sql.Types.VARCHAR);
        ps.setDouble(2, balance);
        if (id != null)
            ps.setString(3, key.id);
        else
            ps.setNull(3, java.sql.Types.VARCHAR);
        int ret = ps.executeUpdate();
        if (ret == 0)
            ex = new RuntimeException("ejbStore: Execute failed");
    }
    catch (Exception e) {
        ex = new RuntimeException(e.getMessage());
    }
    finally {
        try {
            if (con != null) con.close();
            if (ps != null) ps.close();
        }
        catch (Exception e) {
            ex = new RuntimeException(e.getMessage());
        }
    }
    if (ex != null) throw ex;
}

```

```

    }

    catch (EJBException ee) {
        ex = new RuntimeException(ee.getMessage());
    }

    catch (SQLException se) {
        ex = new RuntimeException(se.getMessage());
    }

    try {
        ps.close();
        con.close();
    }

    catch (Exception e) { }

    if (ex != null)
        throw ex;
}

public AccountPK ejbCreate(String id, String ownername, double balance) throws CreateException {
    this.id = id;
    this.ownername = ownername;
    this.balance = balance;
    CreateException ce = null;
    Connection con = null;
    PreparedStatement ps = null;
    AccountPK key = null;
    try {
        con = getConnection();
        ps = con.prepareStatement("insert into olalaDB.dbo.accounts (id, ownername, balance) values (?, ?, ?)");

        if (id != null)
            ps.setString(1, id);
        else
            ps.setNull(1, java.sql.Types.VARCHAR);

        if (ownername != null)
            ps.setString(2, ownername);

```

```

else
    ps.setNull(2, java.sql.Types.VARCHAR);
ps.setDouble(3, balance);
int ret = ps.executeUpdate();
if (ret != 1)
    ce = new CreateException("Create failed");
key = new AccountPK();
key.id = id;
}
catch (EJBException ee) {
    ce = new CreateException(ee.getMessage());
}
catch (SQLException se) {
    ce = new CreateException(se.getMessage());
}
try {
    ps.close();
    con.close();
}
catch (Exception e) { }
if (ce != null)
    throw ce;
return key;
}

public void ejbPostCreate(String id, String ownername, double balance) { }
public void ejbRemove() throws RemoveException {
    RemoveException re = null;
    Connection con = null;
    PreparedStatement ps = null;
    try {
        con = getConnection();
        AccountPK key = (AccountPK)context.getPrimaryKey();
        ps = con.prepareStatement("delete from olalaDB.dbo.accounts where id = ?");
    }

```



```

        ps.setString(1, key.id);

        int result = ps.executeUpdate();

        if (result == 0)

            re = new RemoveException("Remove failed");
    }

    catch (EJBException ee) {

        re = new RemoveException(ee.getMessage());
    }

    catch (SQLException se) {

        re = new RemoveException(se.getMessage());
    }

    try {

        ps.close();

        con.close();

    }

    catch (Exception e) { }

    if (re != null)

        throw re;
}

public AccountPK ejbCreate(String id, String ownername) throws CreateException {

    return ejbCreate(id,ownername,0);
}

public void ejbPostCreate(String id, String ownername) { }

public Enumeration ejbFindByOwnerName(String name) throws FinderException {

    PreparedStatement pstmt=null;

    Connection con=null;

    Vector v=new Vector();

    try{

        con=getConnection();

        pstmt=con.prepareStatement("select id from accounts where ownerName=?");

        pstmt.setString(1,name);

        ResultSet rs=pstmt.executeQuery();

        while (rs.next()) {

```

```

        String id=rs.getString("id");

        v.addElement(new AccountPK(id));
    }

    return v.elements();
}

catch(Exception e) {
    throw new FinderException(e.toString());
}

finally{
    try{
        pstmt.close();
    }

    catch (Exception e) { }

    try {
        con.close();
    }

    catch (Exception e) { }

}

}

public void deposit(double amt) {
    balance+=amt;
}

public void withdraw(double amt) throws AccountException
{
    if (amt>balance){
        throw new AccountException("Your balance is "+balance+"! You cannot withdraw "+amt+"!");
    }

    balance-=amt;
}
}

```

6.3.1.5 Exception Class

เป็นคลาส exception ที่สร้างขึ้นเองขยายมาจาก java.lang.Exception ใ้ค้ดังนี้

AccountException.java

```
package olala;

public class AccountException extends java.lang.Exception{

    public AccountException(){
        super();
    }

    public AccountException(Exception e) {
        super(e.toString());
    }

    public AccountException(String s) {
        super(s);
    }
}
```

6.3.1.6 Deployment Descriptor

bean-managed persistent จะมีไฟล์ descriptor 2 ไฟล์เหมือนกับ session bean คือ ejb-jar.xml และ weblogic-ejb-jar.xml โคลดดังนี้

ejb-jar.xml

```
<?xml version="1.0"?>
<!DOCTYPE ejb-jar PUBLIC "-//Sun Microsystems, Inc.//DTD Enterprise JavaBeans 1.1//EN"
'http://java.sun.com/j2ee/dtds/ejb-jar_1_1.dtd'>
<ejb-jar>
  <enterprise-beans>
    <entity>
      <description></description>
      <ejb-name>Account</ejb-name>
      <home>olala.AccountHome</home>
      <remote>olala.Account</remote>
      <ejb-class>olala.AccountEJB</ejb-class>
      <persistence-type>Bean</persistence-type>
      <prim-key-class>olala.AccountPK</prim-key-class>
      <reentrant>False</reentrant>
      <cmp-field>
```

```

        <field-name>balance</field-name>
    </cmp-field>
    <cmp-field>
        <field-name>ownername</field-name>
    </cmp-field>
    <cmp-field>
        <field-name>id</field-name>
    </cmp-field>
    <resource-ref>
        <res-ref-name>jdbc/OlalaDB</res-ref-name>
        <res-type>javax.sql.DataSource</res-type>
        <res-auth>Container</res-auth>
    </resource-ref>
</entity>
</enterprise-beans>
<assembly-descriptor>
    <container-transaction>
        <method>
            <ejb-name>Account</ejb-name>
            <method-name>*</method-name>
        </method>
        <trans-attribute>Required</trans-attribute>
    </container-transaction>
</assembly-descriptor>
</ejb-jar>

```

weblogic-ejb-jar.xml

```

<?xml version="1.0"?>
<!DOCTYPE weblogic-ejb-jar PUBLIC "-//BEA Systems, Inc.//DTD WebLogic 5.1.0 EJB//EN"
"http://www.bea.com/servers/wls510/dtd/weblogic-ejb-jar.dtd">
<weblogic-ejb-jar>
    <weblogic-enterprise-bean>
        <ejb-name>Account</ejb-name>
    </weblogic-enterprise-bean>
</weblogic-ejb-jar>

```

```

<reference-descriptor>
  <resource-description>
    <res-ref-name>jdbc/OlalaDB</res-ref-name>
    <jndi-name>OlalaDB</jndi-name>
  </resource-description>
</reference-descriptor>
<jndi-name>Account</jndi-name>
</weblogic-enterprise-bean>
</weblogic-ejb-jar>

```

6.3.1.7 คอมไพล์และแพ็คเกจ

คอมไพล์จาวาไฟล์ จัดไฟล์ทั้งหมดอยู่ใน path ดังนี้

```

./Account/olala/Account.class
./Account/olala/AccountEJB.class
./Account/olala/AccountHome.class
./Account/olala/AccountException.class
./Account/olala/AccountPK.class
./Account/meta-inf/ejb-jar.xml
./Account/meta-inf/weblogic-ejb-jar.xml

```

สร้าง jar ไฟล์โดยใช้คำสั่งดังนี้

```
./Account>jar cv0f stdAccount.jar olala meta-inf
```

และทำคำสั่งต่อไปนี้

```
./Account/java weblogic.ejbrc -compiler javac stdAccount.jar Account.jar
```

แล้วจึงนำ Account.jar ไปดีพลอย

6.3.1.8 ไคลเอนต์

เซต classpath ซึ่งไปยัง Account.jar หรือ path ที่มีอินเทอร์เฟซของ bean โค้ดไคลเอนต์ดังนี้

AccountClient.java

```

import olala.*;
import javax.ejb.*;
import javax.naming.*;
import java.util.*;

```

```

public class AccountClient{

public static void main(String[] args){

    try {

        Context ctx = getInitialContext();

        AccountHome home = (AccountHome) ctx.lookup("Account");

        Account account = null;

        try {

            home.create("123-456.7890","John Smith");

            Enumeration enumeration=home.findByName("John Smith");

            if (enumeration!=null){

                account=(Account) enumeration.nextElement();

            }

            else{

                throw new Exception("Could not find account");

            }

            System.out.println("Initial Balance = "+account.getBalance());

            account.deposit(100);

            System.out.println("After depositing 100, account balance = "+account.getBalance());

            AccountPK pk=(AccountPK) account.getPrimaryKey();

            account=null;

            account=home.findByPrimaryKey(pk);

            System.out.println("Found account with ID "+pk+. Balance = "+account.getBalance());

            System.out.println("Now trying to withdraw $150, which is more than is "+"currently available.
This should generate an exception..");

            account.withdraw(150);

        }

        catch (Exception e) {

            e.printStackTrace();

        }

        finally {

            try {

                System.out.println("Destroying account..");

                if (account!=null) {

```

```

        account.remove();
    }
}

catch (Exception e) {
    e.printStackTrace();
}
}

catch (Exception e) {
    e.printStackTrace();
}
}

public static Context getInitialContext() throws NamingException{
    Properties p = new Properties();
    p.put(Context.INITIAL_CONTEXT_FACTORY, "weblogic.jndi.WLInitialContextFactory");
    p.put(Context.PROVIDER_URL, url);
    if (user != null) {
        p.put(Context.SECURITY_PRINCIPAL, user);
        if (password == null)
            password = "";
        p.put(Context.SECURITY_CREDENTIALS, password);
    }
    return new InitialContext(p);
}

static String url = "t3://localhost:7001";
static String user = null;
static String password = null;
}

```

คอมไพล์แล้วรันจะได้ผลดังนี้

Initial Balance = 0.0

After depositing 100, account balance = 100.0

Found account with ID 123-456.7890. Balance = 100.0

Now trying to withdraw \$150, which is more than is currently available. This should generate an

```

exception..

olala.AccountException: Your balance is 100.0! You cannot withdraw 150.0!
at weblogic.rmi.internal.AbstractOutboundRequest.sendReceive
(AbstractOutboundRequest.java:90)
at olala.AccountEJBEOImpl_WLStub.withdraw(AccountEJBEOImpl_WLStub.java:791)
at AccountClient.main(AccountClient.java:53)
Destroying account..

```

ที่เกิด exception ขึ้นเนื่องจากการถอนเงินเกินกว่าที่มี จึงเกิด AccountException ขึ้น

6.4 Container-Managed Persistent Entity Bean

เมธอดที่ต้องอิมพลิเมนต์ของ container-managed persistent ต่างจาก bean-managed persistent ดังตารางที่ 6-2

เมธอด	คำอธิบาย
setEntityContext() (เหมือนกับ bean-managed persistent)	เมื่อคอนเทนเนอร์สร้างอินสแตนซ์ของ bean แล้วจะเรียกเมธอดนี้
ejbFind<...>(<...>)	ไม่จำเป็นต้องอิมพลิเมนต์ finder method คอนเทนเนอร์จะจัดการให้ โดยวิธีในการระบุเพื่อให้คอนเทนเนอร์รู้วิธีแมปนั้นขึ้นอยู่กับผลิตภัณฑ์

ตารางที่ 6-2 คำอธิบายเมธอดใน *Container-Managed Persistent Entity Bean*

เมธอด	คำอธิบาย
EjbCreate(<...>)	เมื่อไคลเอ็นต์เรียกเมธอด create() กับ home interface คอนเทนเนอร์จะเรียกเมธอด ejbCreate() กับอินสแตนซ์ของ bean ที่พูลอยู่ ejbCreate() จะทำหน้าที่ในการสร้างข้อมูลใหม่ในฐานข้อมูลให้ เมธอดนี้เหมือนกับ bean-managed persistent แต่กับ container-managed persistent ไม่จำเป็นต้องอิมพลิเมนต์เมธอดนี้
ejbPostCreate(<...>) (เหมือนกับ bean-managed persistent)	bean class จะต้องกำหนดเมธอด ejbPostCreate() ให้เข้าคู่กับ ejbCreate() คือต้องมีพารามิเตอร์เหมือนกัน คอนเทนเนอร์จะเรียกเมธอดนี้หลังจากเรียก ejbCreate()
ejbActivate() (เหมือนกับ bean-managed persistent)	เมื่อไคลเอ็นต์ต้องการเรียกบิซิเนสเมธอดกับ EJB object แต่ไม่มีอินสแตนซ์ของ entity bean นั้นที่ bind กับ EJB object คอนเทนเนอร์จะนำ bean จากพูลและเปลี่ยนสถานะเป็น ready โดยเมธอดนี้จะไม่ถูกเรียก

	ระหว่างทรานแซกชัน
ejbLoad()	คอนเทนเนอร์จะเรียกเมธอดนี้เพื่อโหลดข้อมูลจากฐานข้อมูลไปยังอินสแตนซ์ของ bean ไม่ต้องอิมพลิเมนต์เมธอดนี้
ejbStore()	คอนเทนเนอร์จะเรียกเมธอดนี้เพื่ออัปเดตข้อมูลจากอินสแตนซ์ของ bean ลงฐานข้อมูล ไม่ต้องอิมพลิเมนต์เมธอดนี้
ejbPassivate() (เหมือนกับ bean-managed persistent)	คอนเทนเนอร์จะเรียกเมธอดนี้เมื่อนำ entity bean เก็บลงในพูล
ejbRemove()	จะลบข้อมูลในฐานข้อมูล ไม่ต้องอิมพลิเมนต์เมธอดนี้
unsetEntityContext() (เหมือนกับ bean-managed persistent)	เมธอดนี้ใช้ในการลบ bean ออกจากเอนไวรอนเมนต์ คอนเทนเนอร์จะเรียกเมธอดนี้ก่อนที่อินสแตนซ์ของ entity bean จะถูกทำลาย

ตารางที่ 6-2 คำอธิบายเมธอดใน *Container-Managed Persistent Entity Bean* (ต่อ)

6.4.1 ตัวอย่างโค้ด Container-Managed Persistent Entity Bean

ตัวอย่างคือ bean ที่เก็บข้อมูลผลิตภัณฑ์ สคริปต์ที่ใช้สร้างเทเบิลดังนี้

create table products (productId varchar(64), name varchar(64), basePrice numeric(18), description varchar(64));

6.4.1.1 Remote Interface

โค้ดดังนี้

Product.java

```
package olala;

import javax.ejb.EJBObject;
import java.rmi.RemoteException;

public interface Product extends EJBObject{

    public String getProductId() throws RemoteException;

    public String getName() throws RemoteException;

    public void setName(String name) throws RemoteException;

    public double getBasePrice() throws RemoteException;

    public void setBasePrice(double basePrice) throws RemoteException;

    public String getDescription() throws RemoteException;

    public void setDescription(String description) throws RemoteException;

}
```

6.4.1.2 Home Interface

โค้ดดังนี้

ProductHome.java

```
package olala;

import javax.ejb.EJBHome;
import javax.ejb.FinderException;
import java.rmi.RemoteException;
import javax.ejb.CreateException;
import java.util.Enumuration;

public interface ProductHome extends EJBHome{

    public Product findByPrimaryKey(String key)    throws FinderException, RemoteException;

    public Product create(String productId, String name, String description, double basePrice)
        throws CreateException, RemoteException;

    public Enumuration findAllProducts()    throws FinderException, RemoteException;

    public Enumuration findCheapProducts(double maxPrice)
        throws FinderException, RemoteException;

    public Enumuration findExpensiveProducts(double minPrice)
        throws FinderException, RemoteException;

    public Enumuration findByBasePrice(double basePrice)
        throws FinderException, RemoteException;

    public Enumuration findByDescription(String description)
        throws FinderException, RemoteException;

    public Enumuration findByName(String name)    throws FinderException, RemoteException;

}
```

6.4.1.3 Enterprise Bean Class

container-managed persistent ช่วยลดการโค้ดติดต่อด้วย JDBC เป็นจำนวนมาก โค้ดดังนี้

ProductEJB.java

```
package olala;

import javax.ejb.EntityBean;
import javax.ejb.EntityContext;
import javax.ejb.CreateException;
```

```

public class ProductEJB implements EntityBean{

    public String productId;

    public String name;

    public double basePrice;

    public String description;

    private EntityContext context;

    public ProductEJB() { }

    public String getProductId() { return(productId); }

    public String getName() { return(name); }

    public void setName(String name) { this.name = name; }

    public double getBasePrice() { return(basePrice); }

    public void setBasePrice(double basePrice) { this.basePrice = basePrice; }

    public String getDescription() { return(description); }

    public void setDescription(String description) { this.description = description; }

    public void setEntityContext(EntityContext ec) { this.context = ec; }

    public void unsetEntityContext() { this.context = null; }

    public void ejbActivate() { }

    public void ejbPassivate() { }

    public void ejbLoad() { }

    public void ejbStore() { }

    public void ejbRemove() { }

    public String ejbCreate(String productId, String name, String description, double basePrice)

        throws CreateException {

        this.productId = productId;

        this.name = name;

        this.basePrice = basePrice;

        this.description = description;

        return null;

    }

    public void ejbPostCreate(String productId, String name, String description, double basePrice) { }

}

```

6.4.1.4 Deployment Descriptor

container-managed persistent จะต้องมีไฟล์ descriptor เพิ่มจากเดิม โดยไฟล์นั้นจะเป็นไฟล์ที่ใช้แมปข้อมูลจากออบเจกต์ไปเป็นข้อมูลในฐานข้อมูล โดยอาจจะมีไฟล์ที่ใช้แมปนั้นมากกว่าหนึ่งไฟล์ก็ได้

ejb-jar.xml

```
<?xml version="1.0"?>
<!DOCTYPE ejb-jar PUBLIC "-//Sun Microsystems, Inc.//DTD Enterprise JavaBeans 1.1//EN"
"http://java.sun.com/j2ee/dtds/ejb-jar_1_1.dtd">
<ejb-jar>
  <enterprise-beans>
    <entity>
      <description></description>
      <ejb-name>Product</ejb-name>
      <home>olala.ProductHome</home>
      <remote>olala.Product</remote>
      <ejb-class>olala.ProductEJB</ejb-class>
      <persistence-type>Container</persistence-type>
      <prim-key-class>java.lang.String</prim-key-class>
      <reentrant>False</reentrant>
      <cmp-field>
        <field-name>description</field-name>
      </cmp-field>
      <cmp-field>
        <field-name>basePrice</field-name>
      </cmp-field>
      <cmp-field>
        <field-name>name</field-name>
      </cmp-field>
      <cmp-field>
        <field-name>productId</field-name>
      </cmp-field>
      <primkey-field>productId</primkey-field>
    </entity>
  </enterprise-beans>
</ejb-jar>
```

```

<assembly-descriptor>
  <container-transaction>
    <method>
      <ejb-name>Product</ejb-name>
      <method-name>*</method-name>
    </method>
    <trans-attribute>Required</trans-attribute>
  </container-transaction>
</assembly-descriptor>
</ejb-jar>

```

weblogic-ejb-jar.xml

```

<?xml version="1.0"?>
<!DOCTYPE weblogic-ejb-jar PUBLIC "-//BEA Systems, Inc.//DTD WebLogic 5.1.0 EJB//EN"
'http://www.bea.com/servers/wls510/dtd/weblogic-ejb-jar.dtd'>
<weblogic-ejb-jar>
  <weblogic-enterprise-bean>
    <ejb-name>Product</ejb-name>
    <persistence-descriptor>
      <persistence-type>
        <type-identifier>WebLogic_CMP_RDBMS</type-identifier>
        <type-version>5.1.0</type-version>
        <type-storage>META-INF/weblogic-cmp-rdbms-jar-Product.xml</type-storage>
      </persistence-type>
      <persistence-use>
        <type-identifier>WebLogic_CMP_RDBMS</type-identifier>
        <type-version>5.1.0</type-version>
      </persistence-use>
    </persistence-descriptor>
    <jndi-name>Product</jndi-name>
  </weblogic-enterprise-bean>
</weblogic-ejb-jar>

```

weblogic-ejb-jar.xml จะแมปข้อมูลโดยใช้ข้อมูลในไฟล์ตามที่ระบุในแท็ก type-storage

weblogic-cmp-rdbms-jar-Product.xml

```
<?xml version="1.0"?>
<!DOCTYPE weblogic-rdbms-bean PUBLIC "-//BEA Systems, Inc.//DTD WebLogic 5.1.0 EJB
RDBMS Persistence//EN" 'http://www.bea.com/servers/wls510/dtd/weblogic-rdbms-persistence.dtd'>
<weblogic-rdbms-bean>
  <pool-name>OlalaPool</pool-name>
  <table-name>products</table-name>
  <attribute-map>
    <object-link>
      <bean-field>description</bean-field>
      <dbms-column>description</dbms-column>
    </object-link>
    <object-link>
      <bean-field>basePrice</bean-field>
      <dbms-column>basePrice</dbms-column>
    </object-link>
    <object-link>
      <bean-field>name</bean-field>
      <dbms-column>name</dbms-column>
    </object-link>
    <object-link>
      <bean-field>productId</bean-field>
      <dbms-column>productId</dbms-column>
    </object-link>
  </attribute-map>
  <finder-list>
    <finder>
      <method-name>findAllProducts</method-name>
      <finder-query>
        <![CDATA[(= 1 1)]]>
      </finder-query>
    </finder>
  </finder-list>
</weblogic-rdbms-bean>
```

```
</finder>

<finder>
  <method-name>findCheapProducts</method-name>
  <method-params>
    <method-param>double</method-param>
  </method-params>
  <finder-query>
    <![CDATA[( < basePrice $0)]]>
  </finder-query>
</finder>

<finder>
  <method-name>findExpensiveProducts</method-name>
  <method-params>
    <method-param>double</method-param>
  </method-params>
  <finder-query>
    <![CDATA[( > basePrice $0)]]>
  </finder-query>
</finder>

<finder>
  <method-name>findByBasePrice</method-name>
  <method-params>
    <method-param>double</method-param>
  </method-params>
  <finder-query>
    <![CDATA[( = basePrice $0)]]>
  </finder-query>
</finder>

<finder>
  <method-name>findByDescription</method-name>
  <method-params>
    <method-param>java.lang.String</method-param>
  </method-params>
```

```

    <finder-query>
      <![CDATA[(= description $0)]]>
    </finder-query>
  </finder>
  <finder>
    <method-name>findByName</method-name>
    <method-params>
      <method-param>java.lang.String</method-param>
    </method-params>
    <finder-query>
      <![CDATA[(= name $0)]]>
    </finder-query>
  </finder>
</finder-list>
</weblogic-rdbms-bean>

```

ไฟล์นี้จะใช้พูลของฐานข้อมูลในการสร้าง connection โดยมีชื่อว่า OlalaPool ที่ระบุในแท็ก pool-name และชื่อเทเบิล products ที่ระบุในแท็ก table-name รวมถึงการแมปข้อมูลจากชื่อของคอลัมน์ไปเป็นชื่อของตัวแปร ส่วน find method จะใช้สคริปต์ในการค้นหา โดยสคริปต์ (= name \$0) หมายถึง where name = พารามิเตอร์ตัวแรก และ (= 1 1) หมายถึงค้นหาทั้งหมด โดยวิธีการนี้จะแตกต่างกันไปตามแต่ละผลิตภัณฑ์

6.4.1.5 คอมไพล์และแพ็คเกจ

คอมไพล์จาวาไฟล์ จัดไฟล์ทั้งหมดอยู่ใน path ดังนี้

```

./Product/olala/Product.class
./Product/olala/ProductEJB.class
./Product/olala/ProductHome.class
./Product/meta-inf/ejb-jar.xml
./Product/meta-inf/weblogic-ejb-jar.xml
./Product/meta-inf/weblogic-cmp-rdbms-jar-Product.xml

```

สร้าง jar ไฟล์โดยใช้คำสั่งดังนี้

```
./Product>jar cv0f stdProduct.jar olala meta-inf
```

และทำคำสั่งต่อไปนี้

./Product/java weblogic.ejbrc -compiler javac stdProduct.jar Product.jar

แล้วจึงนำ Product.jar ไปดีพลอย

6.4.1.6 ไคลเอนต์

เซต classpath ที่ไปยัง Product.jar หรือ path ที่มีอินเทอร์เฟซของ bean โค้ดไคลเอนต์ดังนี้

ProductClient.java

```
import olala.*;

import javax.ejb.*;
import javax.naming.*;
import java.util.*;

public class ProductClient {

    public static void main(String[] args){

        ProductHome home=null;

        try {

            Context ctx = getInitialContext();

            home = (ProductHome) ctx.lookup("Product");

            home.create("123-456.7890","P6.250","350 Mhz Pentium",200);
            home.create("123-456.7891","P5-400","400 Mhz Pentium",300);
            home.create("123-456.7892","P5-450","450 Mhz Pentium",400);
            home.create("123-456.7893","SD-64","64 MB SDRAM",50);
            home.create("123-456.7894","SD-128","128 MB SDRAM",100);
            home.create("123-456.7895","SD-256","256 MB SDRAM",200);

            Enumeration enum=home.findByName("SD-64");

            System.out.println("The following product descriptions match the product name SD-64:");

            while (enum.hasMoreElements()){

                Product prod=(Product) enum.nextElement();

                System.out.println(prod.getDescription());

            }

            System.out.println("Calling finder to find all products that cost $200");

            enum=home.findByBasePrice(200);

            while (enum.hasMoreElements()){

                Product prod=(Product) enum.nextElement();

                System.out.println(prod.getDescription());

            }

        }

    }

}
```

```

    }
}
catch (Exception e) {
    e.printStackTrace();
}
finally{
    if (home!=null){
        try{
            System.out.println("Destroying products..");
            Enumeration enum=home.findAllProducts();
            while (enum.hasMoreElements()){
                try{
                    Product prod=(Product)enum.nextElement();
                    prod.remove();
                }
                catch (Exception e){
                    e.printStackTrace();
                }
            }
        }
        catch (Exception e){
            e.printStackTrace();
        }
    }
}

public static Context getInitialContext() throws NamingException
{
    Properties p = new Properties();
    p.put(Context.INITIAL_CONTEXT_FACTORY, "weblogic.jndi.WLInitialContextFactory");
    p.put(Context.PROVIDER_URL, url);
    if (user != null) {
        p.put(Context.SECURITY_PRINCIPAL, user);
    }
}

```

```

        if (password == null)
            password = "";
        p.put(Context.SECURITY_CREDENTIALS, password);
    }
    return new InitialContext(p);
}

static String url = "t3://localhost:7001";
static String user = null;
static String password = null;
}

```

คอมไพล์แล้วรันได้ผลดังนี้

The following product descriptions match the product name SD-64:

64 MB SDRAM

Calling finder to find all products that cost \$200

350 Mhz Pentium

256 MB SDRAM

Destroying products..

6.5 เปรียบเทียบ BMP กับ CMP Entity Bean

การเลือกกระหว่าง entity bean 2 ชนิดนี้ขึ้นอยู่กับงานที่จะทำ เนื่องจากทั้ง 2 ชนิดนี้มีข้อดีและข้อเสียต่างกัน โดยจะเปรียบเทียบในหัวข้อต่าง ๆ ดังนี้

1) จำนวนโค้ดและเวลาในการพัฒนา

container-managed persistent มีจำนวนโค้ดน้อยกว่าและพัฒนาได้เร็วกว่า เนื่องจากไม่ต้องเขียนโค้ดแมปข้อมูลด้วย JDBC

2) Bugs

การดีบั๊ก container-managed persistent ยากกว่า bean-managed persistent เนื่องจาก bean-managed persistent เป็นการควบคุมโค้ด JDBC ทั้งหมดทำให้ง่ายต่อการตรวจสอบโค้ดเหล่านั้น แต่ container-managed persistent จะช่วยลดการเกิดบั๊กได้มากกว่า bean-managed persistent ที่ต้องเขียนโค้ดเองทั้งหมด

3) การควบคุม

bean-managed persistent ควบคุมผ่านทาง JDBC จึงมีความยืดหยุ่นในการแมปออบเจ็กต์ไปยังฐานข้อมูล สำหรับ container-managed persistent ขึ้นอยู่กับว่าคอนเทนเนอร์รองรับการแมปที่ซับซ้อนแค่ไหน

4) ความเป็นอิสระจากแอปพลิเคชันเซิร์ฟเวอร์และฐานข้อมูล

container-managed persistent เป็นอิสระจากฐานข้อมูล เนื่องจากไม่มีการโค้ด JDBC สามารถจะเปลี่ยนแปลงได้ง่าย โดยอาจเปลี่ยนจากฐานข้อมูลเชิงสัมพันธ์เป็นฐานข้อมูลเชิงวัตถุโดยไม่ต้องแก้ไขโค้ดใด ๆ แต่การแมปจากออบเจ็กต์ไปยังฐานข้อมูลไม่มีมาตรฐานที่แน่นอน จึงไม่ portable กับแอปพลิเคชันเซิร์ฟเวอร์ใด ๆ ส่วน bean-managed persistent นั้น portable กับแอปพลิเคชันเซิร์ฟเวอร์และฐานข้อมูลที่เป็นกลางคือรองรับ SQL มาตรฐาน

5) การเรียนรู้และค่าใช้จ่าย

นักพัฒนาส่วนใหญ่จะมีความเข้าใจในการทำงานกับฐานข้อมูลเชิงสัมพันธ์ ดังนั้นการใช้ bean-managed persistent จึงไม่ต้องการการเรียนรู้ แต่กับ container-managed persistent จะต้องมีการเรียนรู้วิธีการแมปของผลิตภัณฑ์ใด ๆ ที่ใช้ รวมถึงค่าใช้จ่ายในการเรียนรู้วิธีการแมปในแต่ละผลิตภัณฑ์ของแอปพลิเคชันเซิร์ฟเวอร์